

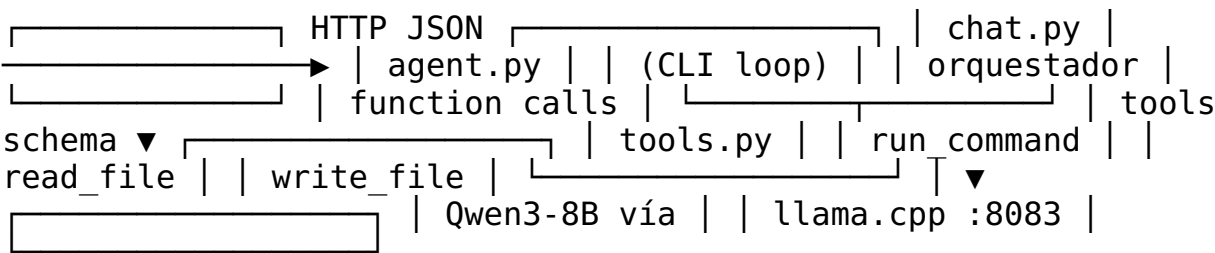
aios-agent v2.0 — Agente SRE con function calling nativo

Agente ligero de Operaciones de Fiabilidad de Sitio (SRE) que usa **function calling nativo** sobre el modelo local **Qwen3-8B** (servido por llama.cpp). Puede ejecutar comandos Linux, leer archivos de configuración/logs y escribir cambios controlados, todo a través de una conversación en español.

¿Qué hace?

- Responde preguntas de sysadmin en español.
- Ejecuta comandos shell en la máquina local (run_command).
- Lee archivos de configuración y logs (read_file).
- Escribe archivos en rutas permitidas, bloqueando directorios de sistema (write_file).
- Mantiene contexto conversacional y realiza hasta 5 turnos de razonamiento tool→LLM.
- Planifica y ejecuta tareas multi-paso sin intervención intermedia.

Arquitectura



- `chat.py`: bucle interactivo.
- `agent.py`: gestiona mensajes, llama al LLM, ejecuta tool calls y devuelve respuestas.
- `tools.py`: definición de herramientas y handlers.

Roadmap

Features implementadas en v2.0 (todas completadas):

- ☒ Function calling nativo sobre llama.cpp server (/v1/chat/completions)
- ☒ Tool run_command: ejecución de comandos shell con timeout, captura de stdout/stderr/exit_code/elapsed
- ☒ Tool read_file: lectura de archivos con control de permisos y límites de tamaño
- ☒ Tool write_file: escritura de archivos con bloqueo de rutas de sistema críticas

- ☐ Bucle conversacional con hasta 5 turnos tool→LLM y contexto persistente por sesión
- ☐ Planificación y ejecución multi-paso sin intervención humana intermedia
- ☐ Seguridad básica: advertencia previa a comandos destructivos y protección de /etc, /boot, /sys, /proc, /dev
- ☐ CLI interactivo en español (chat.py) con comandos salir/exit/quit
- ☐ README.md completo con arquitectura, uso y roadmap
- ☐ Documentación ejecutiva en PDF (docs/ejecutivo.pdf)

Planificación multi-paso

Para tareas complejas el agente no ejecuta un único function call: primero **piensa**, descompone y luego ejecuta los pasos de forma secuencial.

Cómo funciona

1. **Prompt de planificación:** el system prompt instruye al LLM a generar un plan numerado de pasos y a seguir ejecutándolo con la instrucción EJECUTA sin explicar.
2. **MAX_TURNS=5:** el loop de function calling permite hasta 5 turnos, suficiente para tareas de varios pasos sin quedar corto.
3. **Ejecución secuencial:** cada tool call se realiza, se inyecta el resultado en el contexto y el LLM decide el siguiente paso hasta que la tarea termine o se agote el presupuesto de turnos.
4. **Sin intervención humana intermedia:** el modelo ejecuta directamente; el usuario recibe solo el resultado final resumido.

Ejemplo de ejecución

Usuario: instala WordPress con Docker y MariaDB

text Paso 1: Verificar que Docker esté instalado y corriendo.
 Paso 2: Crear red Docker y contenedor MariaDB con variables de entorno. Paso 3: Levantar contenedor WordPress vinculado a MariaDB. Paso 4: Mostrar estado final de contenedores y puertos expuestos.

El agente ejecuta cada paso vía run_command, recibe la salida y avanza automáticamente. Al finalizar responde con el resumen de lo realizado.

Beneficios

- Resuelve tareas compuestas sin fragmentar la consulta del usuario.
- Aprovecha el razonamiento del LLM para ordenar dependencias (primero MariaDB, luego WordPress).
- Mantiene el control del bucle: puede pedir confirmación si detecta un paso destructivo o crítico.

Requisitos

- Python 3.10+
- requests (pip install requests)
- llama.cpp server corriendo Qwen3-8B en `http://localhost:8083/v1/chat/completions`

Uso

`bash python3 chat.py`

Ejemplos de consulta:

````text`

muestra el uso de disco lee /var/log/syslog escribe un script de backup en /tmp/backup.sh reinicia nginx ```

Escribe salir, exit o quit para terminar.

# Seguridad

- Antes de comandos destructivos el modelo advierte y pide confirmación.
- `write_file` bloquea rutas de sistema (/etc, /boot, /sys, /proc, /dev).
- El agente no guarda historial conversacional entre sesiones.

# Archivos

- `agent.py` — orquestador de function calling.
- `tools.py` — herramientas shell, lectura y escritura.
- `chat.py` — interfaz de chat por terminal.
- `README.md` — este documento.
- `CHANGELOG.md` — histórico de cambios.
- `docs/ejecutivo.pdf` — resumen ejecutivo en PDF.

# Historial de versiones

## v2.0 — Agente SRE con function calling nativo sobre Qwen3-8B

- Reescritura completa del repositorio.
- Agente ligero de SRE con function calling nativo vía llama.cpp server.
- Nuevas herramientas:
- `run_command`: ejecuta comandos shell en Linux.
- `read_file`: lee archivos de configuración y logs.
- `write_file`: escribe archivos, bloqueando rutas de sistema críticas.
- Soporte conversacional en español con hasta 5 turnos de razonamiento.
- Planificación y ejecución multi-paso sin intervención intermedia.

- Seguridad básica: advertencia antes de comandos destructivos y bloqueo de `/etc`, `/boot`, `/sys`, `/proc`, `/dev`.
- CLI interactivo en `chat.py`.
- README.md completo con roadmap de features implementadas.
- PDF ejecutivo v2.0 en `docs/ejecutivo.pdf`.

## **Licencia**

MIT / Uso interno.